

Checkpoint2: SLAM

Peilin Meng, Yijie Liao, Yun Jong Na, Yunzhu Deng
University of Michigan

I. OCCUPANCY GRID MAPPING

Occupancy grid mapping provides a discrete spatial representation where each cell (x_g, y_g) represents the probability of an area being occupied.

A. Coordinate Transformation and Interpolation

To maintain map accuracy while the robot moves, it is necessary to convert world-frame coordinates to grid-frame indices and interpolate the sensor pose during a scan acquisition.

1) *World to Grid Transformation*: The mapping from a continuous world coordinate (x_w, y_w) to a discrete grid index (x_g, y_g) is defined by the grid resolution r and the origin coordinates (x_o, y_o) :

$$x_g = \lfloor \frac{x_w - x_o}{r} \rfloor, \quad y_g = \lfloor \frac{y_w - y_o}{r} \rfloor \quad (1)$$

2) *Motion Compensation*: We account for the robot's motion during the time it takes to complete a single laser scan. For each ray i in a scan of N rays, we calculate an interpolation factor $t = i/(N - 1)$. The ray origin P_i and the global heading θ_i are computed as follows:

$$P_i = P_{start} + t \cdot (P_{end} - P_{start}) \quad (2)$$

$$\theta_i = (\psi_{start} + t \cdot \Delta\psi) + \alpha_i \quad (3)$$

where ψ represents the robot's yaw in the world frame and α_i is the angle of the i -th ray in the lidar's local frame. This ensures that each ray is projected from the precise location the robot occupied at that specific timestamp.

B. Raycasting Algorithms

Two algorithms were implemented to identify the grid cells that must be updated as free or occupied based on the sensor readings.

1) *Divide and Step Along Ray*: The step-based approach samples the ray at fixed intervals. Given a range R and a step size s , which is set to $0.5 \cdot r$ to prevent skipping cells), the algorithm computes the position (x_d, y_d) for each step $d = 0, s, 2s, \dots, R$:

$$x_d = x_{origin} + d \cos(\theta), \quad y_d = y_{origin} + d \sin(\theta) \quad (4)$$

The corresponding grid indices are then updated. While intuitive, this requires numerous floating point and trigonometric operations per ray.

2) *Bresenham's Algorithm*: Bresenham's algorithm provides an efficient alternative using integer arithmetic to approximate the line between the origin (x_0, y_0) and the endpoint (x_1, y_1) . The core logic relies on an error term $err = dx - dy$, where $dx = |x_1 - x_0|$ and $dy = |y_1 - y_0|$. For each step, the algorithm decides whether to increment the coordinates based on the sign of $e_2 = 2 \cdot err$:

- 1) If $e_2 > -dy$, the algorithm sets $err = err - dy$ and moves in the x-direction.
- 2) If $e_2 < dx$, it sets $err = err + dx$ and moves in the y-direction.

This loop continues until the endpoint is reached, effectively tracing the ray through the grid without trigonometric calls inside the loop.

C. Algorithm Comparison

The performance and mapping quality of both algorithms were evaluated using experimental data and visual output.

1) *Computational Efficiency*: The execution time is calculated by dividing the total execution time for one scan by the amount of rays. Data is summarized in Table I and shown in Figure 1, which reveal a significant performance gap.

TABLE I. Execution Time: Bresenham vs. Step

Algorithm	Mean (μs)	Min (μs)	Max (μs)
Bresenham	5.76	3.61	7.14
Divide and Step	12.28	8.81	14.19

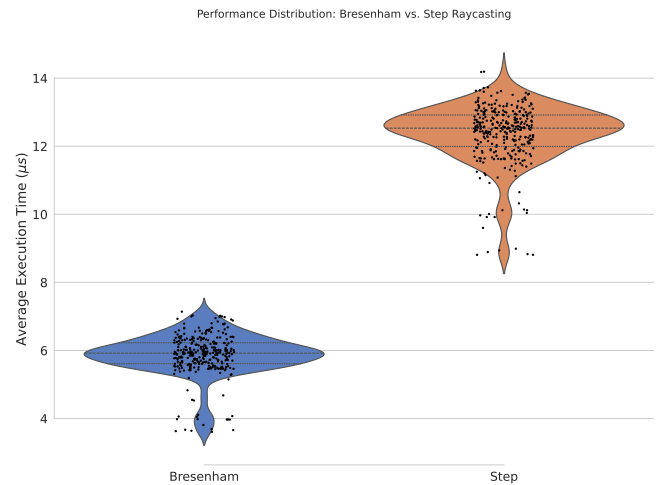
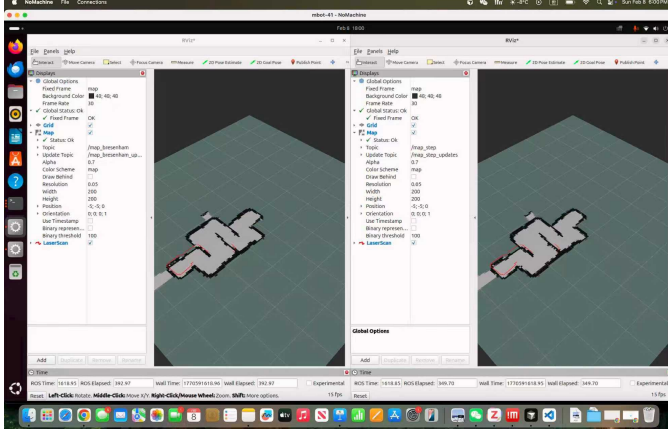


Fig. 1. Performance Distribution: Bresenham vs. Step

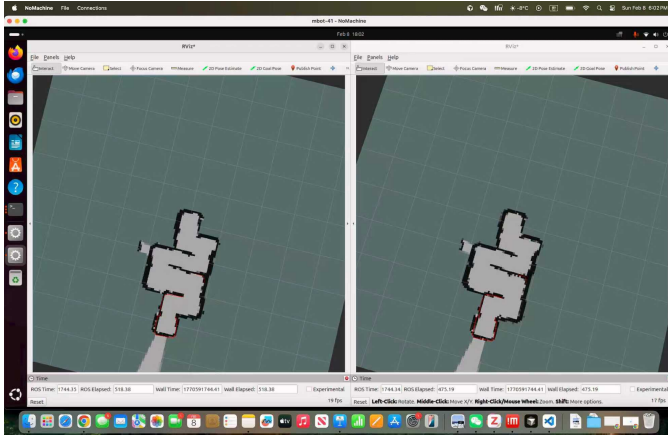
Bresenham's algorithm consistently achieves an average execution time of $5.76 \mu s$ per ray, whereas Divide and

Step requires roughly $12.28 \mu\text{s}$. The violin plot in Figure 1 illustrates that Bresenham is not only faster but also has a tighter distribution, indicating more predictable real time performance.

2) *Mapping Quality*: The occupancy grid mapping comparison visualized via Rviz is shown in Figure 2.



(a) Side View



(b) Bird's-eye View

Fig. 2. Map Comparison: Bresenham (left) vs Step (right)

Both algorithms produce accurate and nearly identical occupancy maps. This confirms that the integer based formulation of Bresenham's algorithm does not introduce additional discretization error or geometric bias compared to the more computationally intensive divide and step method, provided that the step size in the latter is sufficiently small relative to the grid resolution.

D. Discussion

The results of the occupancy grid mapping are consistent with our theoretical expectations.

1) *Mapping Quality*: The generated maps appear clear and structurally accurate, with obstacle boundaries and free space corridors reflecting the actual geometry of the environment. This clarity is largely due to the motion compensation implemented in the ray interpolation phase,

which prevents the geometric blurring that would otherwise occur if all rays in a single scan were projected from a static pose.

2) *Algorithm Comparison*: A visual comparison between the divide and step along ray method and Bresenham's algorithm reveals that their mapping outputs are functionally identical. The results match as long as the step size used in the sampling-based method is sufficiently small relative to the grid resolution, ensuring that no grid cells along the ray are skipped. Under this condition, both methods traverse the same sequence of cells.

The primary difference between the two approaches is computational efficiency rather than spatial accuracy. The divide and step method involves repeated calls to trigonometric functions and floating point multiplications to calculate the coordinates for each step along the ray. In contrast, Bresenham's algorithm operates entirely within the integer domain after the initial setup, using a simple error term and additions to determine the next cell. This explains why Bresenham's algorithm is approximately twice as fast as the divide and step approach.

For a robot operating in real time with high frequency sensor data, the computational savings provided by Bresenham's algorithm make it the superior choice for maintaining a reliable occupancy grid.

II. MONTE CARLO LOCALIZATION

Monte Carlo Localization (MCL) is a probabilistic, particle-filter-based localization algorithm that estimates a robot's pose in a known map using noisy motion and sensor measurements. The posterior belief over robot poses is represented by a set of weighted particles and recursively updated through prediction, correction, and resampling steps.

The implementation of MCL consists of three main components:

- An action (motion) model that propagates particles using odometry information
- A sensor model that evaluates the likelihood of a pose given laser scan measurements and an occupancy grid map
- A particle filter framework including weight normalization, resampling, and pose estimation

All components were implemented using wheel-encoder-based odometry, 2D laser scans, and a pre-built occupancy grid map provided in the lab framework.

A. MCL Action Model

1) *Action Model Overview*: An odometry-based motion model was implemented following the formulation described in Chapter 5 of *Probabilistic Robotics*. The robot motion between two consecutive time steps is decomposed into three components:

- Initial rotation
- Translation

- Final rotation

Independent Gaussian noise is added to each component to account for wheel slip, encoder quantization, and integration drift.

2) *Motion Decomposition*: Given two consecutive odometry poses

$$(x_{t-1}, y_{t-1}, \theta_{t-1}), \quad (x_t, y_t, \theta_t),$$

the motion increments are computed as:

$$\delta_{rot1} = \text{atan2}(y_t - y_{t-1}, x_t - x_{t-1}) - \theta_{t-1} \quad (5)$$

$$\delta_{trans} = \sqrt{(x_t - x_{t-1})^2 + (y_t - y_{t-1})^2} \quad (6)$$

$$\delta_{rot2} = \theta_t - \theta_{t-1} - \delta_{rot1} \quad (7)$$

Each component is corrupted by zero-mean Gaussian noise:

$$\hat{\delta}_{rot1} \sim \mathcal{N}(\delta_{rot1}, \sigma_{rot1}^2) \quad (8)$$

$$\hat{\delta}_{trans} \sim \mathcal{N}(\delta_{trans}, \sigma_{trans}^2) \quad (9)$$

$$\hat{\delta}_{rot2} \sim \mathcal{N}(\delta_{rot2}, \sigma_{rot2}^2) \quad (10)$$

The updated particle pose is then computed as:

$$x' = x + \hat{\delta}_{trans} \cos(\theta + \hat{\delta}_{rot1}) \quad (11)$$

$$y' = y + \hat{\delta}_{trans} \sin(\theta + \hat{\delta}_{rot1}) \quad (12)$$

$$\theta' = \theta + \hat{\delta}_{rot1} + \hat{\delta}_{rot2} \quad (13)$$

3) *Action Model Parameters and Empirical Tuning*: The action model noise parameters were tuned empirically to balance short-term odometry accuracy and long-term particle diversity. In particular, we investigated how different levels of rotational noise affect particle dispersion during sharp turns, as well as the alignment of the estimated trajectory (blue) with the rosbag-recorded reference and odometry trajectories (green and yellow). During parameter tuning, the number of particles was fixed at 800 to isolate the effect of the action model parameters.

Four parameter configurations were evaluated, starting from the original parameters provided in the skeleton code and progressively reducing the rotation noise coefficient. The translation-related parameters were kept fixed to isolate the effect of rotational uncertainty.

Table II summarizes the final parameter values selected for the experiments.

TABLE II. Final Action Model Noise Parameters

Parameter	Description	Value
k_1	Rotation noise coefficient	0.004
k_2	Translation noise coefficient	0.01
min_trans	Minimum translation noise	0.005
min_rot	Minimum rotation noise	0.04

Figure 3 compares the estimated pose trajectories obtained using different action model parameters. The blue trajectory denotes the estimated pose, while the reference and odometry paths are shown for comparison.

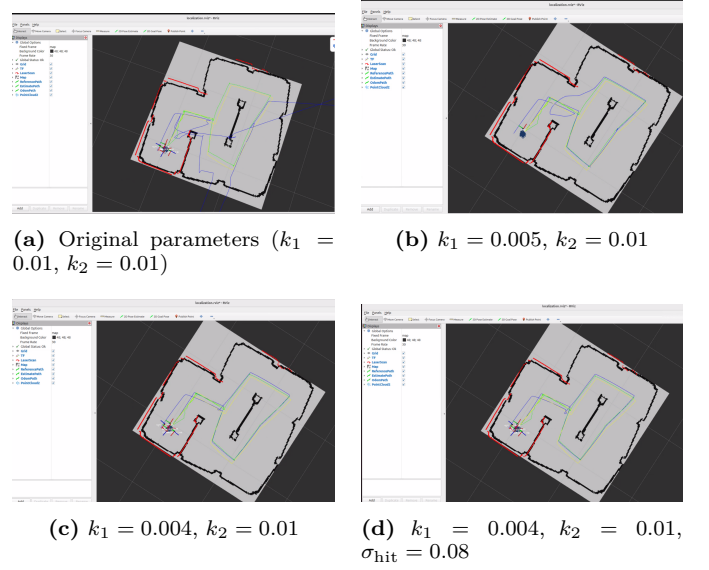


Fig. 3. Effect of Action Model Parameter Tuning on Localization Performance

With the original parameters, particles exhibited excessive dispersion during large rotations, leading to temporary localization degradation. Reducing the rotation noise coefficient improved trajectory smoothness and allowed the estimated path to better align with both the odometry and reference trajectories.

The final parameter set achieved stable tracking throughout most of the trajectory. Although brief particle divergence was observed during sharp turns, the filter successfully recovered once sufficient sensor information became available.

B. Sensor Model and Particle Filter

1) *Sensor Model Overview*: A likelihood field sensor model was implemented based on Chapter 6 of *Probabilistic Robotics*. For each laser ray, the endpoint in the map frame is computed and evaluated using a precomputed distance field. The likelihood decreases exponentially with the distance to the nearest occupied cell.

2) *Measurement Likelihood*: For a laser ray whose endpoint is at distance d from the nearest obstacle, the hit probability is computed as:

$$p_{hit} = \exp\left(-\frac{d^2}{2\sigma^2}\right) \quad (14)$$

A mixture model is used to handle unexpected measurements:

$$p(z | x) = w_{hit} \cdot p_{hit} + w_{rand} \cdot p_{rand} \quad (15)$$

where p_{rand} is a uniform distribution over the sensor range. The total particle likelihood is computed as the product of likelihoods over a subset of laser rays to improve runtime performance.

The standard deviation of the hit model was set to $\sigma = 0.08$, which provided a good balance between noise tolerance and map consistency.

3) *Particle Filter Implementation*: The particle filter follows the standard prediction–correction–resampling pipeline:

- 1) Propagate particles using the action model
- 2) Compute importance weights using the sensor model
- 3) Normalize particle weights
- 4) Resample particles using systematic (low-variance) resampling
- 5) Estimate robot pose as the weighted mean of particles

Systematic resampling was chosen to reduce variance and avoid particle deprivation compared to naive resampling.

C. Performance Evaluation

1) *Particle Filter Runtime*: The average update time of the particle filter was measured for different numbers of particles on a Raspberry Pi. Table III reports the observed average update time per iteration.

TABLE III. Particle Filter Update Time

Number of Particles	Update Time (ms)
100	≈ 13
300	≈ 36
500	≈ 58
1000	≈ 100

To maintain real-time operation at 10 Hz, the particle filter update must complete within a 100 ms time budget. Empirically, the update time scales approximately linearly with the number of particles, i.e.,

$$T(N) \approx aN + b. \quad (16)$$

Using the measured runtimes (approximately 13 ms at $N = 100$ and 58 ms at $N = 500$), we estimate

$$a \approx \frac{58 - 13}{500 - 100} = 0.1125 \text{ ms/particle}$$

$$b \approx 13 - 0.1125 \cdot 100 = 1.75 \text{ ms.}$$

Solving $T(N) \leq 100$ ms yields an estimated upper bound

$$N_{\max} \approx \frac{100 - b}{a} \approx \frac{98.25}{0.1125} \approx 870.$$

This estimate is consistent with our qualitative observations: using $N = 800$ during parameter tuning produced stable and accurate tracking, while $N = 1000$ often led to missed deadlines due to runtime jitter and system overhead (measured updates occasionally exceeding 100 ms). Therefore, although the theoretical limit is around 870 particles, we choose $N = 800$ as a practical operating point that satisfies the 10 Hz constraint with a safety margin.

2) *Localization Accuracy*: The particle filter was evaluated using the provided ROS bag data. Figure 4 compares the estimated robot trajectory with the reference pose trajectory.

The estimated trajectory closely follows the reference path for most of the run. Transient divergence was observed during sharp turns, where particles temporarily spread before reconverging once sufficient sensor information became available.

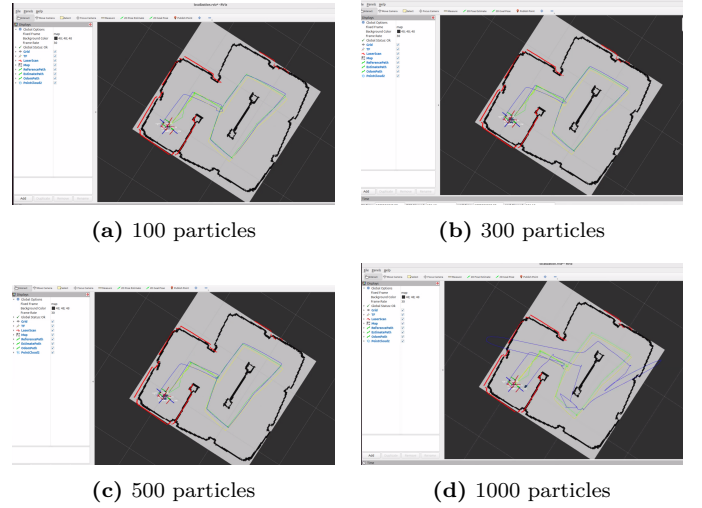


Fig. 4. Estimated Pose Trajectories for Different Particle Counts. The estimated trajectory is shown in blue, the reference trajectory in green, and the odometry trajectory in yellow.

D. Discussion

The odometry-based action model provides accurate short-term motion prediction but accumulates drift over time. The likelihood field sensor model effectively corrects this drift when distinctive map features are present.

Increasing the number of particles improves robustness during ambiguous motion but significantly increases computational cost. Overall, the implemented MCL system demonstrates a clear trade-off between localization accuracy and real-time performance, and achieves reliable localization suitable for downstream navigation tasks.

III. SIMULTANEOUS LOCALIZATION AND MAPPING (SLAM)

This section describes the implementation and integration of a particle filter-based SLAM system that combines Monte Carlo Localization (MCL) with occupancy grid mapping. The system simultaneously estimates the robot's pose while building a map of the environment.

A. SLAM Algorithm Overview

The SLAM algorithm follows a simple iterative approach:

- 1) Use the first laser scan to initialize particles and construct an initial map.
- 2) For each subsequent laser scan:
 - Localize using the map from the previous iteration.
 - Update the map using the pose estimate from localization.

This creates a feedback loop where better maps lead to better localization, and better localization leads to better maps.

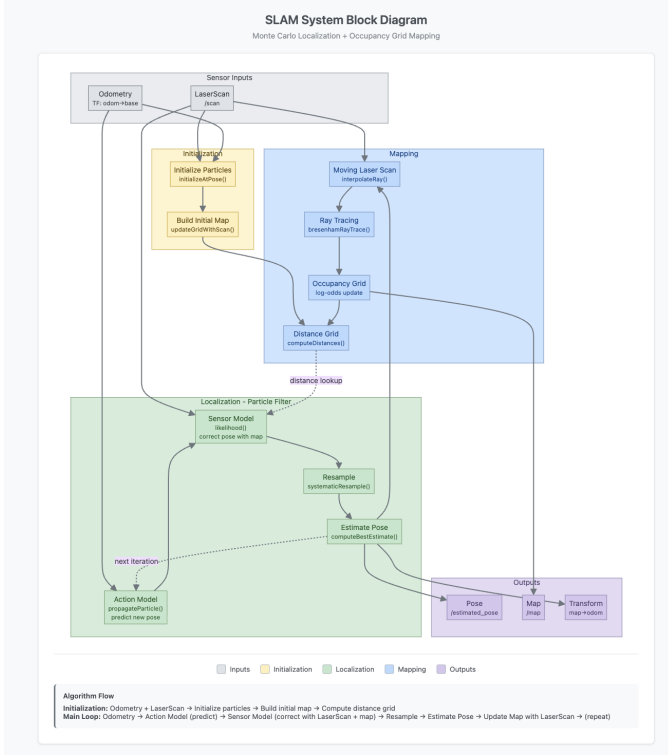


Fig. 5. Block Diagram of The SLAM System Architecture. Inputs (LaserScan and Odometry) feed into initialization, followed by the particle filter localization loop (Sensor Model → Resample → Estimate → Action Model) and mapping updates.

B. System Architecture

Figure 5 shows the block diagram of the SLAM system components and their interactions.

The system consists of the following major components:

- **Sensor Inputs:** LaserScan (/scan) and Odometry (TF: odom → base_footprint)
- **Particle Filter:** Maintains 800 particle hypotheses of the robot pose
- **Sensor Model:** Computes likelihood of each particle given laser observations
- **Action Model:** Propagates particles using odometry with added noise
- **Occupancy Grid:** Log-odds based probabilistic map representation
- **Distance Grid:** Pre-computed obstacle distances for fast sensor model lookups

C. Component Interactions

The SLAM system follows the Bayes filter framework with two key update steps:

1) *Sensor Model (Posterior Update):* The sensor model updates the belief using the current laser observation:

$$\text{Bel}(x_t) = \eta P(z_t|x_t) \text{Bel}^-(x_t) \quad (17)$$

where $\text{Bel}^-(x_t)$ is the prior belief, $P(z_t|x_t)$ is the observation likelihood, and η is a normalizing constant.

The likelihood is computed using the likelihood field model:

$$P(z_t|x_t) = Z_{\text{hit}} \cdot \exp\left(-\frac{d^2}{2\sigma^2}\right) + Z_{\text{rand}} \cdot p_{\text{uniform}} \quad (18)$$

where d is the distance from the ray endpoint to the nearest obstacle (obtained from the distance grid), $\sigma = 0.15$ m, $Z_{\text{hit}} = 0.95$, and $Z_{\text{rand}} = 0.05$.

2) *Action Model (Prior Prediction):* The action model predicts the new prior for the next timestep:

$$\text{Bel}^-(x_{t+1}) = \int P(x_{t+1}|u_t, x_t) \text{Bel}(x_t) dx_t \quad (19)$$

The motion model uses odometry decomposed into $(\delta_{\text{rot1}}, \delta_{\text{trans}}, \delta_{\text{rot2}})$ with Gaussian noise proportional to the motion magnitude.

3) *Resampling and Pose Estimation:* After weighting particles with the sensor model, systematic resampling is performed to focus computational resources on high-probability regions. The final pose estimate is computed as the weighted average of the top 10% of particles, using circular mean for orientation to handle angle wraparound.

4) *Mapping Update:* Using the estimated pose, the occupancy grid is updated:

- **Ray Tracing:** Bresenham's algorithm traces each laser ray through the grid
- **Log-Odds Update:** Cells along the ray receive -0.4 (free), endpoints receive $+0.85$ (occupied)
- **Distance Grid:** BFS flood-fill recomputes distances after each map update

The updated distance grid is then used by the sensor model in the next iteration, completing the feedback loop.

D. Key Parameters

Table IV summarizes the key parameters used in the SLAM system.

TABLE IV. SLAM System Parameters

Parameter	Value	Description
NUM_PARTICLES	800	Number of particles
σ_{hit}	0.15 m	Sensor model std dev
$Z_{\text{hit}}/Z_{\text{rand}}$	0.95 / 0.05	Likelihood weights
k_1 / k_2	0.0015 / 0.0008	Motion noise coefficients
Hit log-odds	+0.85	Occupied cell update
Miss log-odds	-0.4	Free cell update
Grid resolution	0.02 m	Map cell size

E. Results

Figure 6 shows the occupancy grid map generated by the SLAM system while navigating the checkpoint maze.

F. Discussion

The SLAM system successfully generates accurate maps while maintaining consistent pose estimates. Compared to odometry-only mapping (Task 2.1), the SLAM-generated map exhibits sharper wall boundaries and reduced drift over long trajectories. This improvement is attributed to

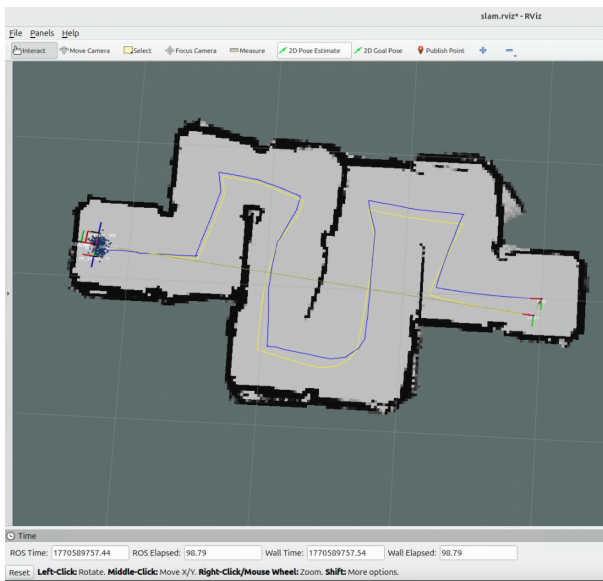


Fig. 6. Occupancy Grid Map Generated by The SLAM System. The map shows clear wall boundaries with minimal drift, demonstrating successful simultaneous localization and mapping.

the particle filter correcting odometry errors using laser scan observations before updating the map.

Key observations:

- The feedback loop between localization and mapping prevents error accumulation
- High map resolution (0.02 m) enables precise sensor model scoring
- Log-odds bounds (± 20) create “sticky” walls resistant to noise
- The distance grid enables efficient $O(1)$ lookups for the sensor model

The system provides stable pose estimates suitable for downstream navigation tasks.